GymFitnessDB

Gym & Fitness Tracker System

Group Name:	Ali Hassan & Muhammad Huzaifa
Course:	Database Management (DBMS) — SE Group A
Document:	Version Control & Milestone Report
Milestones:	M2 | M3 | M4 | M5
Filename:	AliHassan_Huzaifa_Version_Control_DBLab.pdf

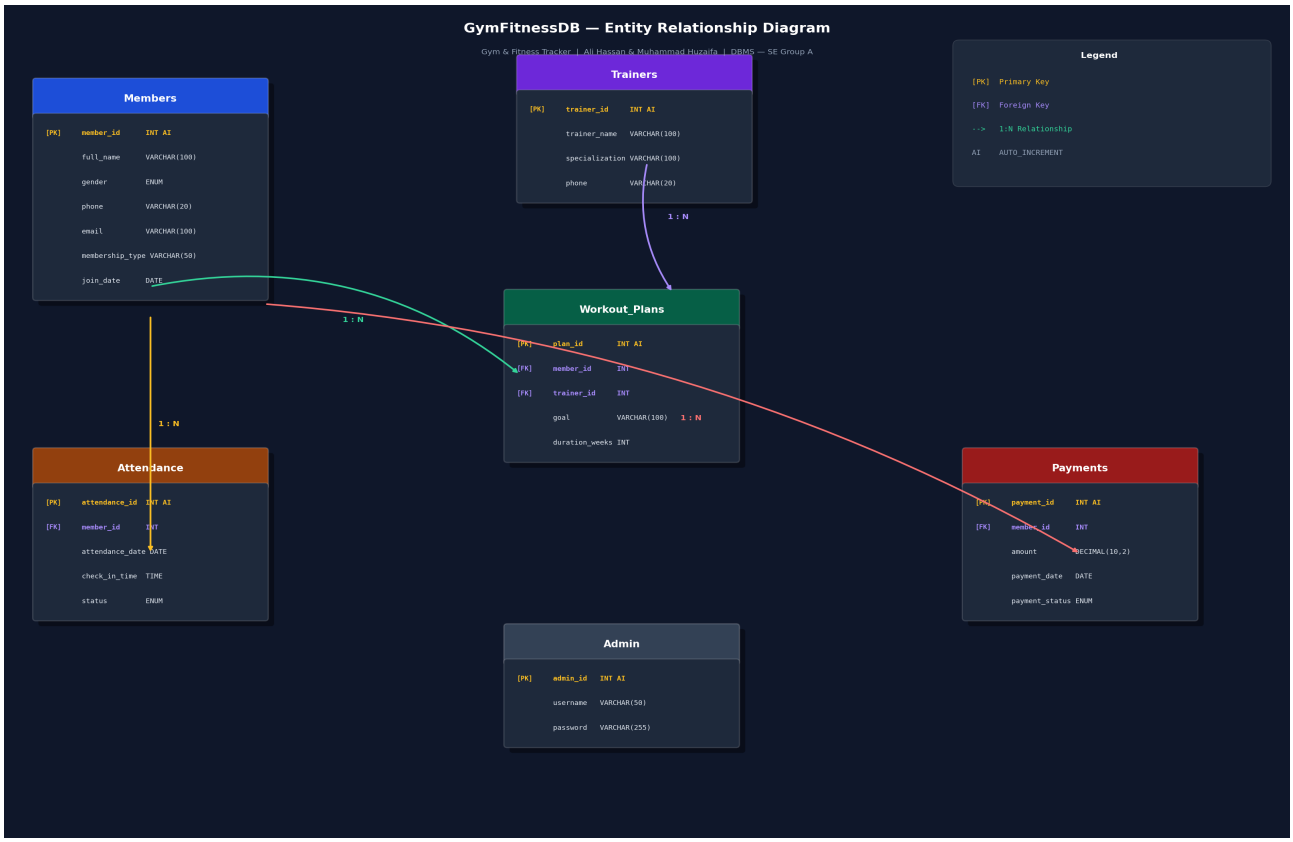
Version Control History

Version	Date	Description	Author
0.1	25-Apr-2026	Initial project proposal and ER diagram draft	Huzaifa Khan
1.0	26-Apr-2026	Final ERD and initial schema completed	Huzaifa Khan
1.1	13-May-2026	Flask web app built (login, members, attendance, payments)	Ali Hassan
2.0	17-May-2026	Normalization (1NF-3NF), updated ERD, dataflow, DDL, DML	Ali Hassan & Huzaifa

Milestone 2 — ERD Design & Normalization

Entity Relationship Diagram (Updated)

The ERD below reflects the fully normalized schema after applying 1NF through 3NF.



Normalization Walkthrough

Every table in GymFitnessDB was reviewed and normalized through 1NF, 2NF, and 3NF. Where a table already satisfied a normal form, a written justification is provided. Where changes were required, the issue, action taken, and reason are documented.

Table: Members

Normal Form	Analysis & Justification
1NF	Already satisfied. All columns store single, atomic values (one name, one phone, one gender per row). No repeating groups or m
2NF	Already satisfied. The primary key is a single column (member_id), so partial dependency is impossible by definition. Every attrib
3NF	Already satisfied. No non-key column depends on another non-key column. full_name, gender, phone, email, membership_type,

Table: Trainers

Normal Form	Analysis & Justification
1NF	Already satisfied. trainer_name, specialization, phone are all atomic and single-valued per row.
2NF	Already satisfied. Single-column PK (trainer_id) makes partial dependency impossible.
3NF	Already satisfied. All attributes directly describe the trainer. specialization does not determine phone or vice versa.

Table: Workout_Plans

Normal Form	Analysis & Justification
1NF	Issue: The goal field could potentially store multiple goals as a comma-separated list. Action: Enforced single-value constraint —
2NF	Already satisfied. PK is plan_id (single column). member_id and trainer_id are foreign keys; goal and duration_weeks describe th
3NF	Already satisfied. goal and duration_weeks are properties of the specific plan record, not of the trainer or member alone. No tran

Table: Attendance

Normal Form	Analysis & Justification
1NF	Issue: No explicit status column existed — the presence of a row was used to imply attendance, which is an implicit encoding. Ac
2NF	Already satisfied. PK is attendance_id. All columns depend directly on it.
3NF	Already satisfied. check_in_time does not determine attendance_date or vice versa — each attendance_id independently determ

Table: Payments

Normal Form	Analysis & Justification
1NF	Already satisfied. All columns — member_id, amount, payment_date, payment_status — are atomic and single-valued.
2NF	Already satisfied. PK is payment_id (single column). No partial dependencies possible.
3NF	Already satisfied. payment_status does not depend on amount or payment_date. amount does not transitively depend on membe

Commit Message

```
M2: Applied 1NF-3NF normalization, added status column to Attendance, updated ERD and schema
```

Milestone 3 — Dataset Preprocessing & Dataflow

Synthetic Data Summary

CSV File	Table	Rows	Generation Tool	Notes
members.csv	Members	100	Python Faker	Realistic Pakistani names, phones, emails
trainers.csv	Trainers	15	Python Faker	10 specialization types
workout_plans.csv	Workout_Plans	80	Python Faker	10 goal types, 4-24 week durations
attendance.csv	Attendance	100	Python Faker	85% Present / 15% Absent ratio
payments.csv	Payments	100	Python Faker	Amounts tied to membership type

Dataflow Description

The GymFitnessDB system manages data through three entry points: the Flask admin web interface (real-time form submissions), bulk CSV import (for initial population), and direct SQL scripts (for setup and validation). Data flows through the system in a dependency-ordered sequence:

Step	Table	Depends On	What Happens
1	Admin	None	Admin logs in; session created; dashboard loads
2	Members	None	New member registered via web form or CSV import
3	Trainers	None	Trainer records loaded independently
4	Workout_Plans	Members + Trainers	Plans created linking each member to a trainer
5	Attendance	Members	Check-in recorded with date, time, and Present/Absent status
6	Payments	Members	Payment amount and status recorded for each member

Outputs from the system include: dashboard statistics (COUNT from Members, SUM from Payments, daily COUNT from Attendance), the members list (JOIN across Members and Trainers via Workout_Plans), attendance logs, payment logs, and validation query results for integrity checking.

Commit Message

```
M3: Synthetic data generated (50-100 rows/table); dataflow documented
```

Milestone 4 — Database Setup (DDL)

Final Normalized Schema

```
Members(member_id PK AI, full_name NOT NULL, gender ENUM NOT NULL, phone UNIQUE NOT NULL,
email UNIQUE, membership_type NOT NULL, join_date NOT NULL) Trainers(trainer_id PK AI,
trainer_name NOT NULL, specialization, phone UNIQUE NOT NULL) Workout_Plans(plan_id PK AI,
member_id FK, trainer_id FK, goal, duration_weeks CHECK>0) Attendance(attendance_id PK AI,
member_id FK, attendance_date NOT NULL, check_in_time, status ENUM DEFAULT 'Present')
Payments(payment_id PK AI, member_id FK, amount CHECK>0, payment_date NOT NULL, payment_status
ENUM) Admin(admin_id PK AI, username UNIQUE NOT NULL, password NOT NULL)
```

Indexes Added

Index Name	Table	Column	Purpose
idx_members_membership	Members	membership_type	Fast filter by membership type
idx_wp_member	Workout_Plans	member_id	Speed up JOINs on member
idx_wp_trainer	Workout_Plans	trainer_id	Speed up JOINs on trainer
idx_att_member	Attendance	member_id	Fast attendance lookup by member
idx_att_date	Attendance	attendance_date	Fast daily attendance queries
idx_pay_member	Payments	member_id	Speed up payment queries by member
idx_pay_status	Payments	payment_status	Fast filter of paid/pending

Commit Message

```
M4: DDL scripts added, EER diagram verified
```

Milestone 5 — Data Population (DML)

Data Loading Method

Data was loaded using LOAD DATA INFILE from the prepared CSV files. INSERT statements are also provided in data_population.sql for systems where LOAD DATA INFILE is restricted. Both approaches produce identical results.

UPDATE & DELETE Operations

Operation	SQL Statement	Purpose
UPDATE	`UPDATE Members SET membership_type='Annual' WHERE member_id=3`	Change a member's plan
UPDATE	`UPDATE Payments SET payment_status='Paid' WHERE payment_id=3 AND payment_status='Pending'`	Mark pending as paid
DELETE	`DELETE FROM Attendance WHERE status='Absent' AND attendance_date < DATE_SUB(CURDATE(), INTERVAL 2`	Clean old absent records
DELETE	`DELETE FROM Workout_Plans WHERE plan_id=5`	Remove cancelled plan

Validation Queries & Expected Output

Validation Check	Query	Expected Result
Row count per table	SELECT COUNT(*) FROM Members; (repeat for each table)	100 Members, 15 Trainers, 80 Plans, 100 Attendance, 100 Pay
NULL check on key columns	SELECT COUNT(*) FROM Members WHERE full_name IS NULL OR join_date IS NULL	0 (no NULL in required fields)
FK integrity — Workout_Plans	LEFT JOIN Members WHERE member_id IS NULL	0 orphan records
FK integrity — Attendance	LEFT JOIN Members WHERE member_id IS NULL	0 orphan records
FK integrity — Payments	LEFT JOIN Members WHERE member_id IS NULL	0 orphan records

Commit Message

```
M5: Data populated, validation queries added
```

GitHub Repository

Repository URL: https://github.com/[your-username]/GymFitnessDB

The repository contains all DDL scripts, DML scripts, CSV files, normalization documentation, dataflow description, updated ERD, and a clear commit history with at least one commit per milestone.

Repository Structure

```
GymFitnessDB/ app.py Flask web application README.md Full project documentation sql/schema.sql M4: DDL (CREATE TABLE statements) sql/data_population.sql M5: LOAD DATA + INSERT + UPDATE + DELETE sql/validation.sql M5: COUNT, NULL, FK validation queries csv/members.csv M3: 100 rows synthetic data csv/trainers.csv M3: 15 rows synthetic data csv/workout_plans.csv M3: 80 rows synthetic data csv/attendance.csv M3: 100 rows synthetic data csv/payments.csv M3: 100 rows synthetic data docs/NORMALIZATION.md M2: 1NF-3NF with justifications docs/DATAFLOW.md M3: Dataflow description docs/ERD.png M2: Updated ER Diagram templates/ Flask HTML templates static/ CSS and images
```